

BD Services

Background Verification API

Complete Integration Documentation
Version 1.0

Table of Contents

1.	Introduction & Overview
2.	Getting Started
3.	Authentication
4.	API Endpoints
5.	Education Verification Check
6.	Address Verification Check
7.	Company Verification Check
8.	Check Status API
9.	Download Report API
10.	Response Codes & Status
11.	Error Handling
12.	Integration Examples

1. Introduction & Overview

The BD Services API enables seamless integration of our background verification portal with your existing platforms. Clients can now submit verification requests, upload documents, track check status, and retrieve reports directly through API calls, eliminating the need for manual portal access.

Key Features:

- Submit Education, Address, and Company verification checks via API
- Track real-time status of submitted checks using Check ID
- Upload supporting documents and candidate details programmatically
- Download verification reports automatically
- Comprehensive error handling and validation
- Secure API authentication and data encryption

2. Getting Started

Base URL:

```
https://api.bdservices.in/v1
```

Protocol: HTTPS (All requests must be over secure connection)

Response Format: JSON

3. Authentication

All API requests require authentication using your unique API Key. Each request must include the API Key in the request headers.

Header	Value
Authorization	Bearer {YOUR_API_KEY}
Content-Type	application/json

Example Request Header:

```
Authorization: Bearer abc123xyz789  
Content-Type: application/json
```

4. API Endpoints Summary

Endpoint	Method	Purpose
/checks/education	POST	Submit Education Verification
/checks/address	POST	Submit Address Verification
/checks/company	POST	Submit Company Verification
/checks/{checkId}/status	GET	Get Check Status
/checks/{checkId}/upload	POST	Upload Documents/Details
/checks/{checkId}/report	GET	Download Verification Report

5. Education Verification Check

Submit a request for education verification.

Endpoint:

POST /checks/education

Request Body - Required & Optional Fields:

Field Name	Type	Required	Description
Request Id	String	Yes	Unique identifier for this request
Candidate Name	String	Yes	Full name of the candidate
College Name	String	No	Name of the college/institution
University / Board	String	Yes	University or Board name
Registration Number	String	No	Registration/Roll number
Degree	String	Yes	Degree obtained (B.Tech, B.A., etc.)
Major / Specialization	String	Yes	Major subject or specialization
Degree Duration From	String	Yes	Start date (YYYY-MM-DD)
Degree Duration To	String	Yes	End date (YYYY-MM-DD)
Graduation Date	String	Yes	Graduation date (YYYY-MM-DD)
Country	String	No	Country of the institution
Affiliation	String	No	UGC/AICTE affiliation status
Platform	String	No	Source platform name
Employee Id	String	No	Internal employee ID
Employee Name	String	No	Employee name
Certificate Image	File	No	Degree certificate image (Base64 or file upload)

Example Request:

```
POST /checks/education HTTP/1.1 Host: api.bdservices.in Authorization: Bearer YOUR_API_KEY
Content-Type: application/json { "request_id": "REQ_EDU_2024_001", "candidate_name": "John Doe",
"college_name": "XYZ College of Engineering", "university_board": "University of Delhi",
"registration_number": "2020001234", "degree": "B.Tech", "major_specialization": "Computer
Science", "degree_duration_from": "2020-07-15", "degree_duration_to": "2024-06-30",
"graduation_date": "2024-06-30", "country": "India", "affiliation": "AICTE", "platform":
"OurPlatform", "employee_id": "EMP_12345", "employee_name": "Jane Smith", "certificate_image":
"base64_encoded_image_data" }
```

Example Response (Success - 201):

```
{ "status": "success", "data": { "check_id": "CHK_EDU_20240615_001", "request_id":
"REQ_EDU_2024_001", "candidate_name": "John Doe", "verification_type": "education", "created_at":
"2024-06-15T10:30:00Z", "current_status": "initiated" }, "message": "Education verification check
submitted successfully" }
```

6. Address Verification Check

Submit a request for address verification.

Endpoint:

POST /checks/address

Request Body - Required & Optional Fields:

Field Name	Type	Required	Description
Request Id	String	Yes	Unique identifier for this request
Candidate Name	String	Yes	Full name of the candidate
Address	String	Yes	Complete residential address
Father Name	String	No	Father's name for verification
State	String	Yes	State/Province
City	String	Yes	City/Town
Period of Stay From	String	Yes	Start date at this address (YYYY-MM-DD)
Period of Stay To	String	Yes	End date at this address (YYYY-MM-DD)
Priority	String	No	Priority level (low/medium/high)

Example Request:

```
POST /checks/address HTTP/1.1 Host: api.bdservices.in Authorization: Bearer YOUR_API_KEY
Content-Type: application/json { "request_id": "REQ_ADDR_2024_001", "candidate_name": "John Doe",
"address": "123 Main Street, Apartment 4B", "father_name": "Robert Doe", "state": "Delhi", "city":
"New Delhi", "period_of_stay_from": "2022-01-15", "period_of_stay_to": "2024-06-30", "priority":
"high" }
```

Example Response (Success - 201):

```
{ "status": "success", "data": { "check_id": "CHK_ADDR_20240615_001", "request_id":
"REQ_ADDR_2024_001", "candidate_name": "John Doe", "verification_type": "address", "created_at":
"2024-06-15T10:35:00Z", "current_status": "initiated" }, "message": "Address verification check
submitted successfully" }
```

7. Company Verification Check

Submit a request for employment/company verification.

Endpoint:

POST /checks/company

Request Body - Required & Optional Fields:

Field Name	Type	Required	Description
Request Id	String	Yes	Unique identifier for this request
Candidate Name	String	Yes	Full name of the candidate
Company Name	String	Yes	Name of the company
Company Address	String	Yes	Company office address
Period of Employment From	String	Yes	Employment start date (YYYY-MM-DD)
Period of Employment To	String	Yes	Employment end date (YYYY-MM-DD)

Example Request:

```
POST /checks/company HTTP/1.1 Host: api.bdservices.in Authorization: Bearer YOUR_API_KEY
Content-Type: application/json { "request_id": "REQ_COMP_2024_001", "candidate_name": "John Doe",
"company_name": "Tech Solutions Pvt. Ltd.", "company_address": "456 Business Park, Tech City",
"period_of_employment_from": "2020-03-01", "period_of_employment_to": "2024-05-31" }
```

Example Response (Success - 201):

```
{ "status": "success", "data": { "check_id": "CHK_COMP_20240615_001", "request_id":
"REQ_COMP_2024_001", "candidate_name": "John Doe", "verification_type": "company", "created_at":
"2024-06-15T10:40:00Z", "current_status": "initiated" }, "message": "Company verification check
submitted successfully" }
```

8. Check Status API

Retrieve the current status of a verification check using the Check ID. The status provides real-time information about the progress of the verification.

Endpoint:

```
GET /checks/{checkId}/status
```

Path Parameters:

Parameter	Type	Description
checkId	String	Unique Check ID returned from submission

Example Request:

```
GET /checks/CHK_EDU_20240615_001/status HTTP/1.1 Host: api.bdservices.in Authorization: Bearer YOUR_API_KEY
```

Example Response (200):

```
{ "status": "success", "data": { "check_id": "CHK_EDU_20240615_001", "verification_type": "education", "candidate_name": "John Doe", "current_status": "processing", "status_timeline": { "initiated": { "status": true, "date": "2024-06-15T10:30:00Z" }, "document_received": { "status": true, "date": "2024-06-15T10:45:00Z" }, "under_review": { "status": true, "date": "2024-06-15T11:00:00Z" }, "insufficiency_raised": { "status": false, "date": null }, "verified": { "status": false, "date": null }, "cancelled": { "status": false, "date": null } }, "estimated_completion": "2024-06-22T18:00:00Z", "last_updated": "2024-06-15T14:30:00Z" } }
```

Possible Status Values:

- **Initiated** - Check has been submitted and is queued for processing
- **Missing Information** - Additional information or documents are required
- **Insufficiency** - Current documents are insufficient; more details needed
- **Processing** - Check is actively being verified
- **Verified** - Check completed and verified successfully
- **Cancelled** - Check has been cancelled

9. Upload Documents & Details API

Upload supporting documents or additional candidate details for an ongoing verification check.

Endpoint:

POST /checks/{checkId}/upload

Path Parameters:

checkId - Unique Check ID returned from initial submission

Request Body:

Field	Type	Required	Description
document_type	String	Yes	Type: certificate, id_proof, address_proof, employment_letter
document_name	String	Yes	Name of the document
file_content	File/Base64	Yes	Document file (Base64 encoded or multipart)
additional_details	String	No	Any additional comments or information

Example Request:

```
POST /checks/CHK_EDU_20240615_001/upload HTTP/1.1 Host: api.bdservices.in Authorization: Bearer YOUR_API_KEY Content-Type: application/json { "document_type": "certificate", "document_name": "degree_certificate.pdf", "file_content": "base64_encoded_pdf_content...", "additional_details": "Updated degree certificate with marks sheet" }
```

Example Response (200):

```
{ "status": "success", "data": { "check_id": "CHK_EDU_20240615_001", "document_id": "DOC_2024_001", "document_type": "certificate", "upload_timestamp": "2024-06-15T11:00:00Z" }, "message": "Document uploaded successfully" }
```

10. Download Report API

Download the final verification report once the check is completed. The report contains verification results and detailed findings.

Endpoint:

GET /checks/{checkId}/report

Query Parameters (Optional):

Parameter	Type	Default	Description
format	String	pdf	Response format: pdf, json, or xml

Example Request:

GET /checks/CHK_EDU_20240615_001/report?format=pdf HTTP/1.1 Host: api.bdservices.in
Authorization: Bearer YOUR_API_KEY

PDF Report Response (200):

Content-Type: application/pdf
Response: Binary PDF file
Header: Content-Disposition: attachment; filename='CHK_EDU_20240615_001_report.pdf'

JSON Report Format Example:

```
{ "status": "success", "data": { "report_id": "REPORT_2024_001", "check_id": "CHK_EDU_20240615_001", "verification_type": "education", "candidate_name": "John Doe", "verification_date": "2024-06-22", "verification_status": "verified", "findings": { "degree": "B.Tech - Verified", "university": "University of Delhi - Verified", "graduation_year": "2024 - Verified", "specialization": "Computer Science - Verified" }, "recommendation": "Candidate education credentials verified. All documents authentic.", "generated_date": "2024-06-22T15:30:00Z" } }
```

11. Response Codes & HTTP Status

Status Code	Meaning	Description
200	OK	Request successful; returned data as expected
201	Created	Request successful; resource created (check submitted)
400	Bad Request	Invalid request syntax or missing required fields
401	Unauthorized	API key is missing, invalid, or expired
403	Forbidden	Authenticated but not authorized for this resource
404	Not Found	Specified resource (check ID, report) does not exist
409	Conflict	Request conflicts with current state (already processed)
422	Unprocessable Entity	Validation failed; semantic errors in request
429	Too Many Requests	Rate limit exceeded; retry after specified time
500	Internal Server Error	Server encountered an unexpected error
503	Service Unavailable	Server temporarily unavailable; retry later

12. Error Handling

All error responses return a JSON object with error details. Implement proper error handling in your client to handle these responses gracefully.

Error Response Format:

```
{ "status": "error", "error_code": "VALIDATION_ERROR", "message": "Validation failed", "details":  
  { "field": "candidate_name", "error": "Candidate name is required" }, "timestamp":  
  "2024-06-15T10:30:00Z" }
```

Common Error Codes:

- **INVALID_API_KEY** - API key is missing or invalid
- **VALIDATION_ERROR** - Required field missing or invalid data format
- **DUPLICATE_REQUEST** - Request ID already exists
- **CHECK_NOT_FOUND** - Check ID does not exist
- **RATE_LIMIT_EXCEEDED** - Too many requests; implement exponential backoff
- **INTERNAL_ERROR** - Server error; retry with exponential backoff
- **INVALID_DOCUMENT** - Uploaded document format not supported

Recommended Error Handling Strategy:

1. Validate all request data before sending
2. Always check HTTP status codes
3. Parse error responses and log error codes
4. Implement exponential backoff for 5xx errors
5. Retry failed requests with appropriate delays
6. Alert support/operations on repeated failures

13. Integration Examples

Example 1: Node.js Integration

```
const axios = require('axios'); const API_KEY = 'your_api_key_here'; const BASE_URL = 'https://api.bdservices.in/v1'; async function submitEducationCheck(data) { try { const response = await axios.post(`${BASE_URL}/checks/education`, data, { headers: { 'Authorization': `Bearer ${API_KEY}`, 'Content-Type': 'application/json' } }); return response.data; } catch (error) { console.error('Error:', error.response?.data || error); throw error; } } async function getCheckStatus(checkId) { try { const response = await axios.get(`${BASE_URL}/checks/${checkId}/status`, { headers: { 'Authorization': `Bearer ${API_KEY}` } }); return response.data; } catch (error) { console.error('Error:', error.response?.data || error); throw error; } } // Usage const eduData = { request_id: 'REQ_EDU_2024_001', candidate_name: 'John Doe', university_board: 'University of Delhi', degree: 'B.Tech', major_specialization: 'CS', graduation_date: '2024-06-30' }; submitEducationCheck(eduData).then(result => getCheckStatus(result.data.check_id)).then(status => console.log('Status:', status)).catch(err => console.error('Failed:', err));
```

Example 2: Python Integration

```
import requests import json API_KEY = 'your_api_key_here' BASE_URL = 'https://api.bdservices.in/v1' def submit_education_check(data): headers = { 'Authorization': f'Bearer {API_KEY}', 'Content-Type': 'application/json' } try: response = requests.post(f'{BASE_URL}/checks/education', headers=headers, json=data) response.raise_for_status() return response.json() except requests.exceptions.RequestException as e: print(f'Error: {e}') raise def get_check_status(check_id): headers = { 'Authorization': f'Bearer {API_KEY}' } response = requests.get(f'{BASE_URL}/checks/{check_id}/status', headers=headers) response.raise_for_status() return response.json() # Usage data = { 'request_id': 'REQ_EDU_2024_001', 'candidate_name': 'John Doe', 'university_board': 'University of Delhi', 'degree': 'B.Tech', 'major_specialization': 'CS', 'graduation_date': '2024-06-30' } result = submit_education_check(data) print(f"Check ID: {result['data']['check_id']}") status = get_check_status(result['data']['check_id']) print(f"Status: {status['data']['current_status']}")
```

Example 3: cURL Commands

```
# Submit Education Check curl -X POST https://api.bdservices.in/v1/checks/education \ -H "Authorization: Bearer YOUR_API_KEY" \ -H "Content-Type: application/json" \ -d '{ "request_id": "REQ_EDU_2024_001", "candidate_name": "John Doe", "university_board": "University of Delhi", "degree": "B.Tech", "major_specialization": "CS", "graduation_date": "2024-06-30" }' # Get Check Status curl -X GET https://api.bdservices.in/v1/checks/CHK_EDU_20240615_001/status \ -H "Authorization: Bearer YOUR_API_KEY" # Download Report curl -X GET \ 'https://api.bdservices.in/v1/checks/CHK_EDU_20240615_001/report' \ -H "Authorization: Bearer YOUR_API_KEY" \ --output report.pdf
```

14. Integration Best Practices

Security:

- Always use HTTPS for all API calls
- Never expose API keys in client-side code
- Rotate API keys periodically
- Use environment variables to store credentials

Error Handling:

- Implement comprehensive try-catch blocks
- Handle all HTTP status codes appropriately
- Log errors for debugging and monitoring
- Provide user-friendly error messages

Performance:

- Implement request queuing to avoid rate limits
- Cache verification status where appropriate
- Use connection pooling for repeated requests
- Set appropriate timeouts for API calls

Data Validation:

- Validate all input data before sending to API
- Ensure date formats are YYYY-MM-DD
- Verify required fields are present
- Sanitize user inputs before submission

Monitoring & Logging:

- Log all API requests and responses
- Monitor API response times
- Set up alerts for failures or rate limits
- Track verification check completion rates